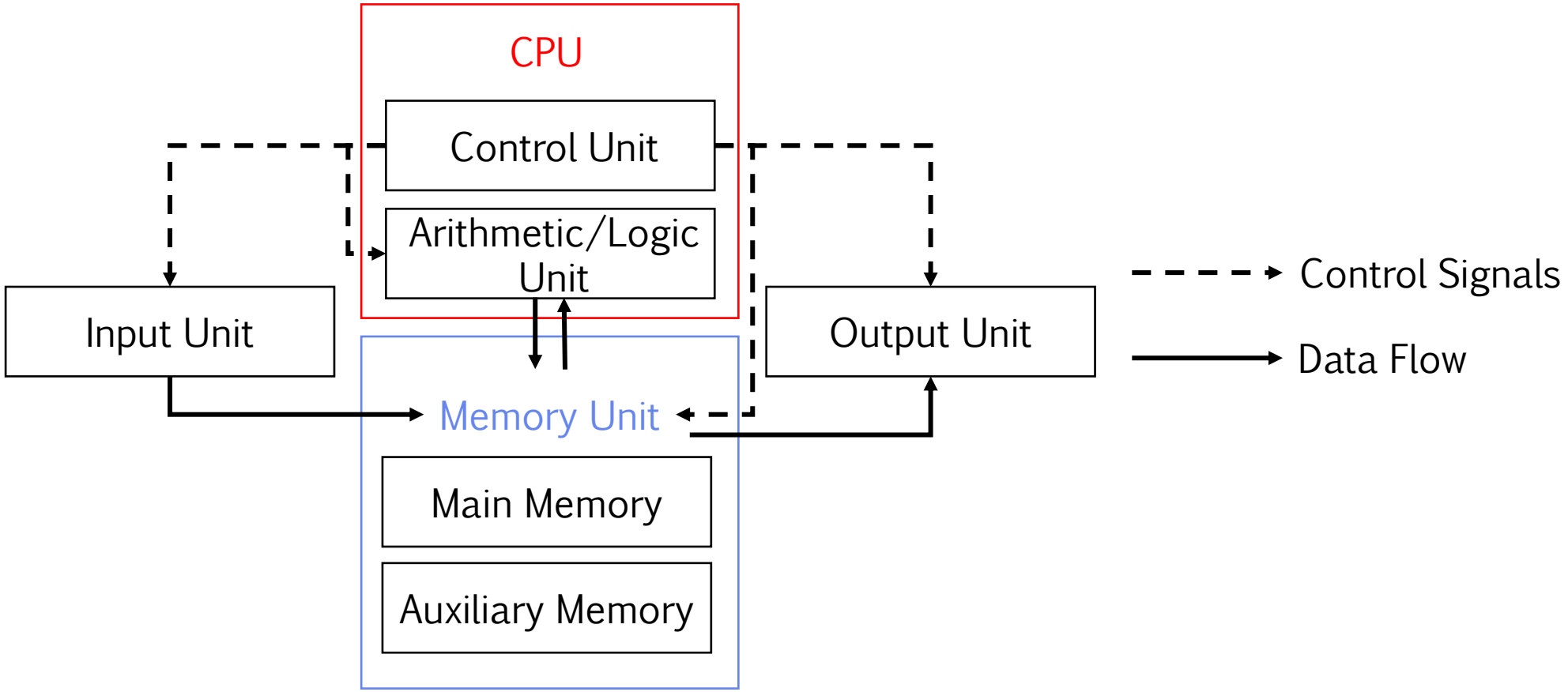
Data Manipulation

Chia-Yeh Hsieh (謝佳燁)



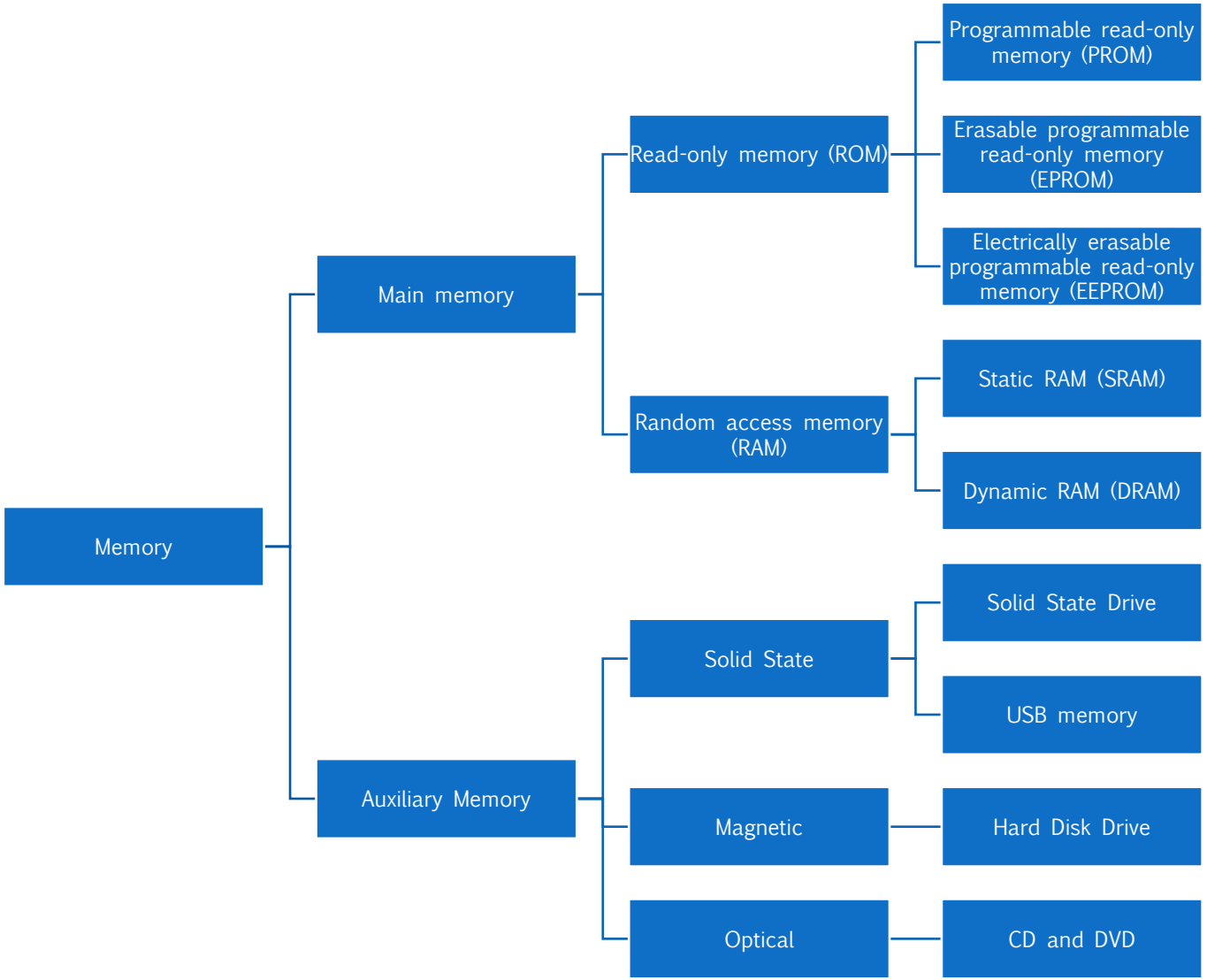
Block Diagram of Computer



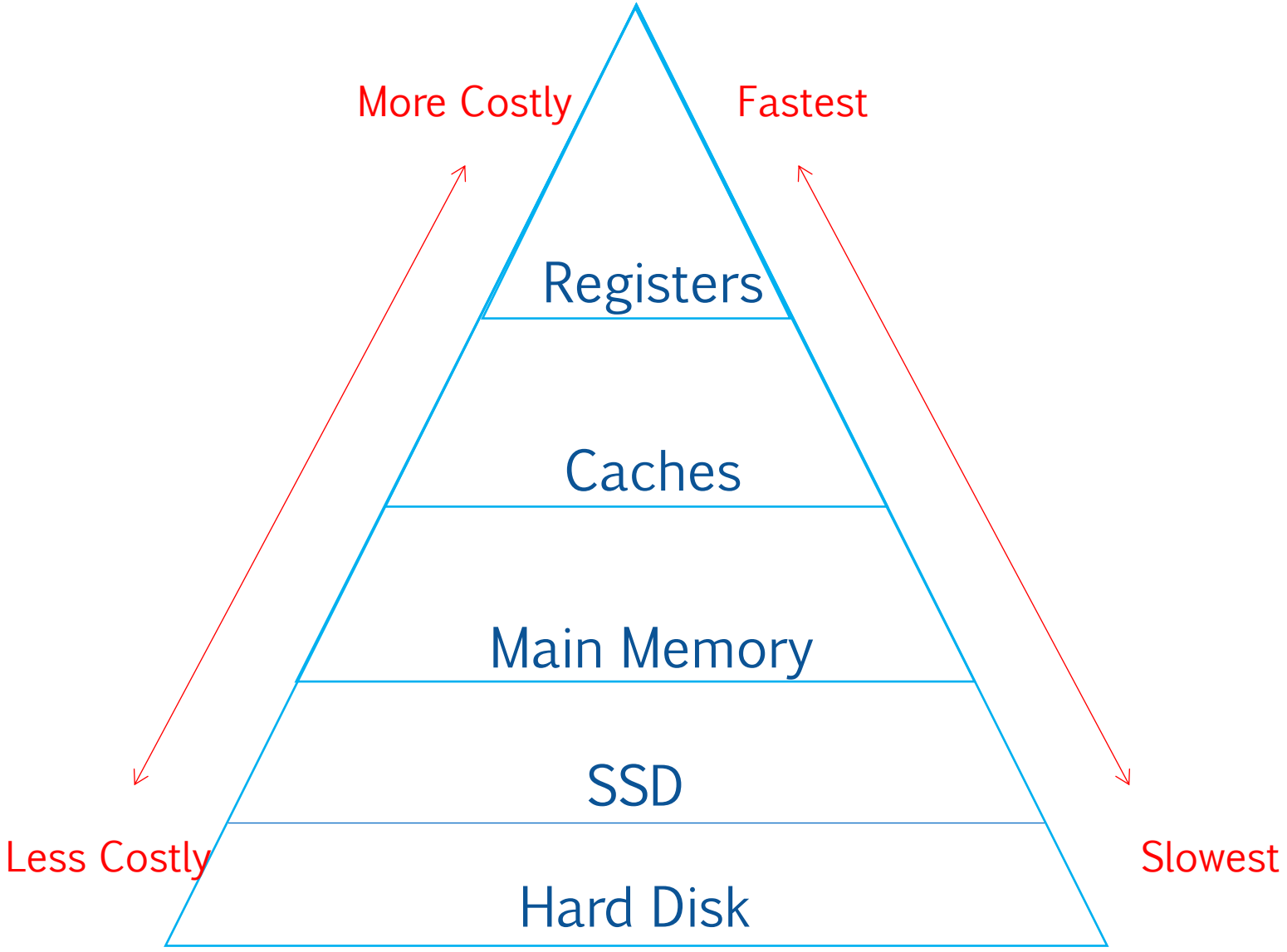
- Three components
  - CPU, Memory Unit, and I/O Units
- Four components
  - CPU, Memory Unit, Input Unit, and Output Unit
  - Control Unit, Arithmetic/Logic Unit, Memory Unit, and I/O Units
- Five components
  - Control Unit, Arithmetic/Logic Unit, Memory Unit, Input Unit, and Output Unit

- Computer Architecture
  - Central Processing Unit (CPU)
    - Control Unit (CU)
      - The control unit controls the operation of each subunit. Controlling is achieved through signals sent from the control unit to other subunits.
    - Arithmetic/Logic Unit (ALU)
      - A digital circuit used to perform arithmetic and logic operations as per the instruction set architecture.
    - Registers and Cache Memory
  - Input Unit
    - keyboard, scanner, mouse
  - Output Unit
    - monitors, printers, speakers, headphones
  - Memory Unit
    - Main memory and auxiliary memory

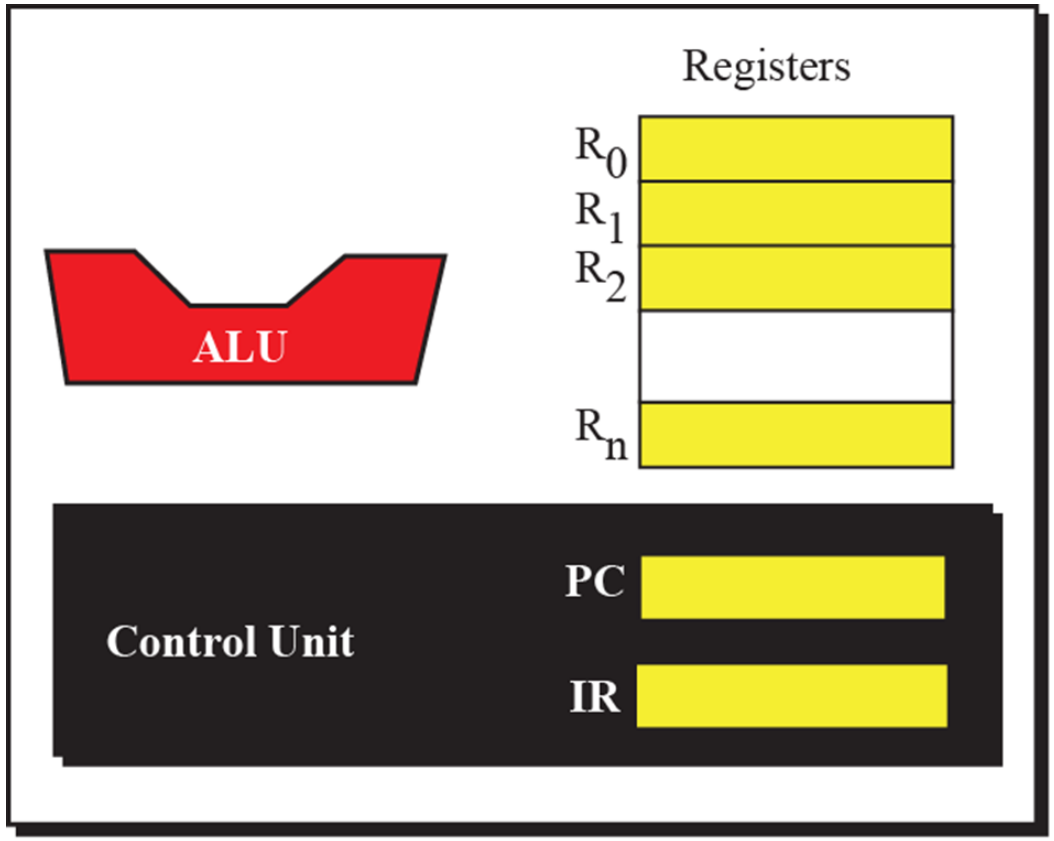
Memory Unit


Memory
Main memory
Read-only memory (ROM)
Programmable read-only memory (PROM)
Erasable programmable read-only memory (EPROM)
Electrically erasable programmable read-only memory (EEPROM)
Random access memory (RAM)
Static RAM (SRAM)
Dynamic RAM (DRAM)
Auxiliary Memory
Solid State
Solid State Drive
USB memory
Magnetic
Hard Disk Drive
Optical
CD and DVD

Memory hierarchy



Central Processing Unit (CPU)

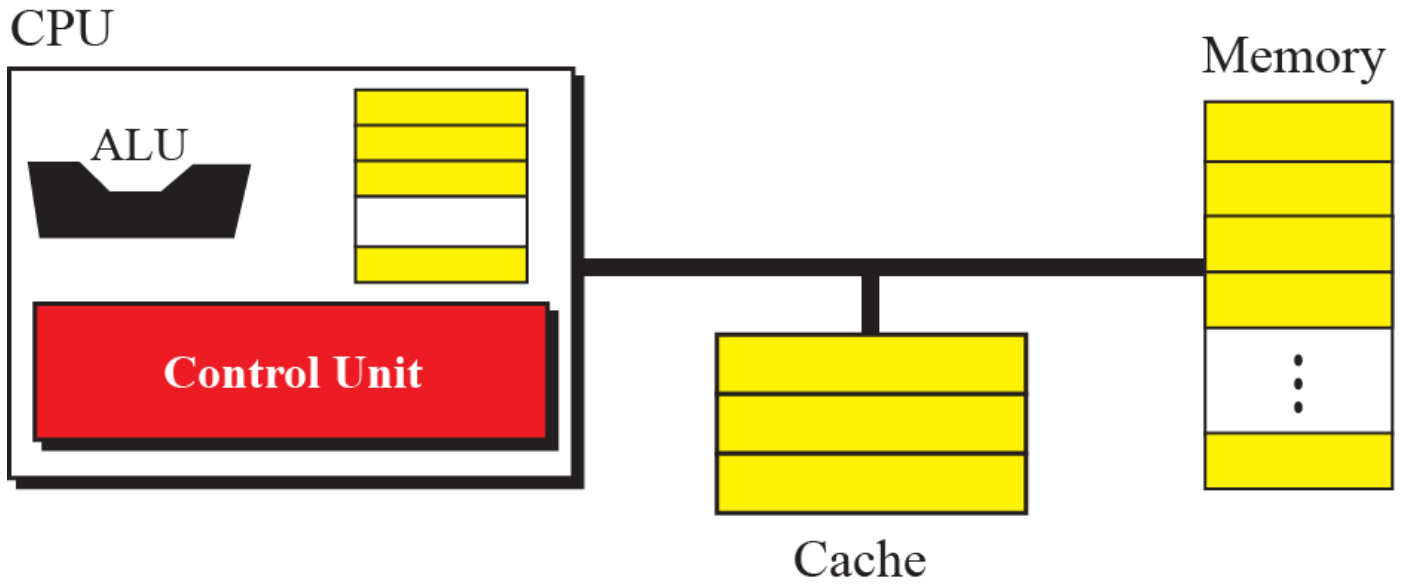


Central Processing Unit (CPU)

- Registers
  - fast stand-alone storage locations that hold data temporarily
  - Multiple registers are needed to facilitate the operation of the CPU
    - Data registers
    - Instruction register
    - Program counter

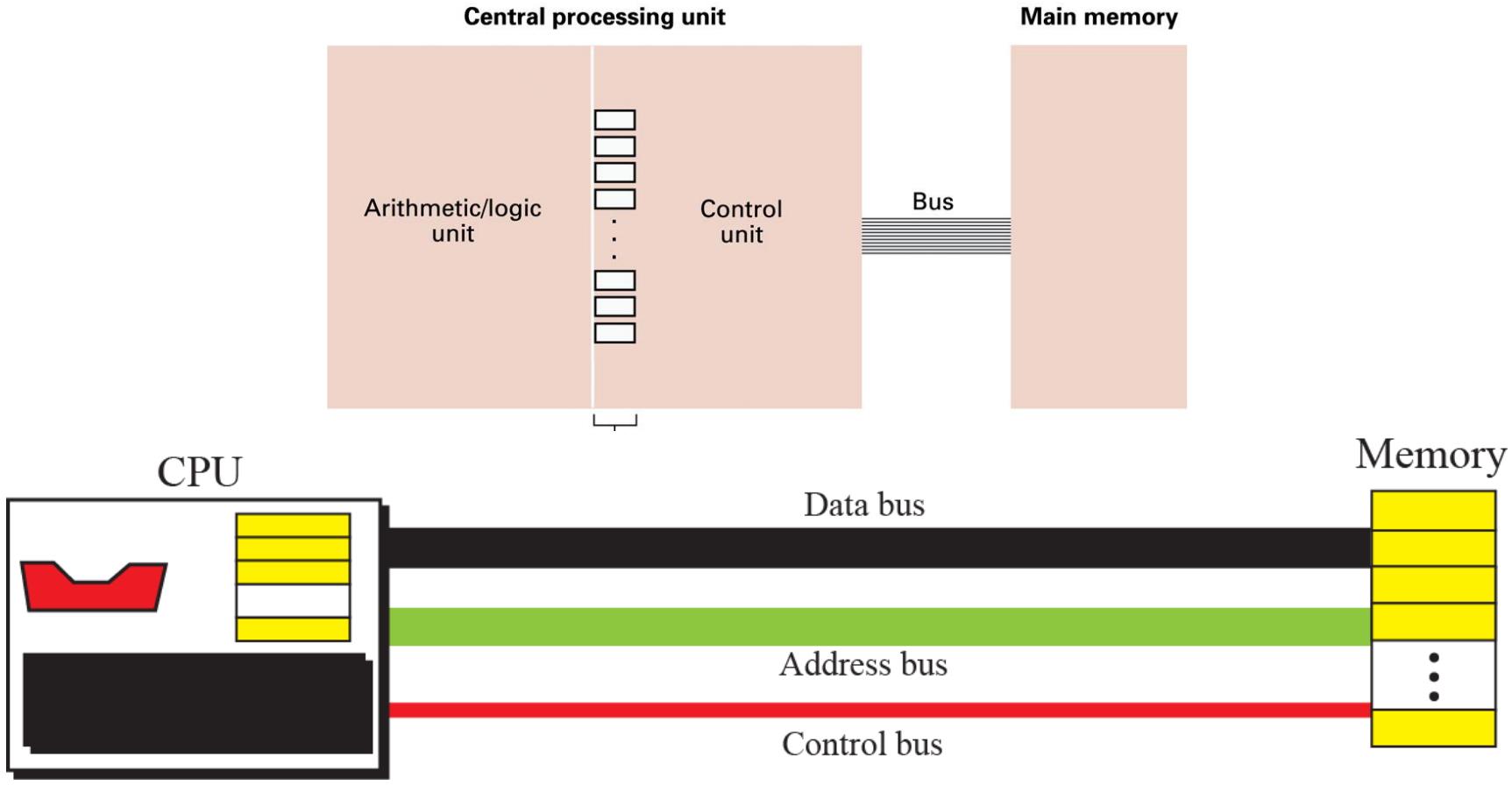
Cache memory

- Cache memory is faster than main memory, but slower than the CPU and its registers
  - Normally small in size
  - Be placed between the CPU and main memory

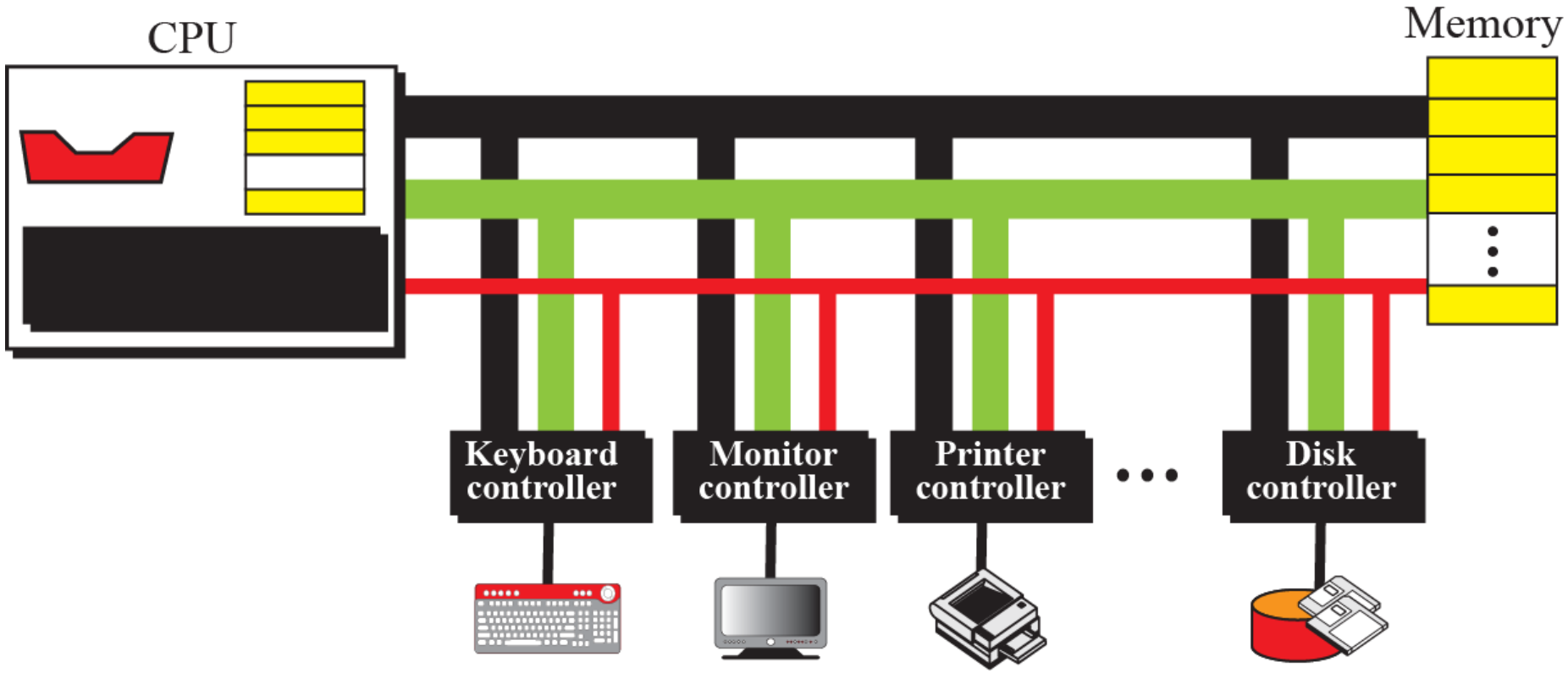


Interconnection

› CPU and main memory connected via a bus



› Connecting I/O devices to the buses



Machine language

- **Machine instruction**: an instruction coded as a bit pattern directly recognizable by the CPU
- **Machine language**: the set of all instructions recognized by a machine

- Machine instruction types
  - Data Transfer: copy data between CPU and main memory
  - Arithmetic/Logic: use existing data values to compute a new data value
  - Control: direct the execution of the program

› Adding values stored in memory

Step 1. Get one of the values to be added from memory and place it in a register.

Step 2. Get the other value to be added from memory and place it in another register.

Step 3. Activate the addition circuitry with the registers used in Steps 1 and 2 as inputs and another register designated to hold the result.

Step 4. Store the result in memory.

Step 5. Stop.

› Dividing values stored in memory

Step 1. LOAD a register with a value from memory.

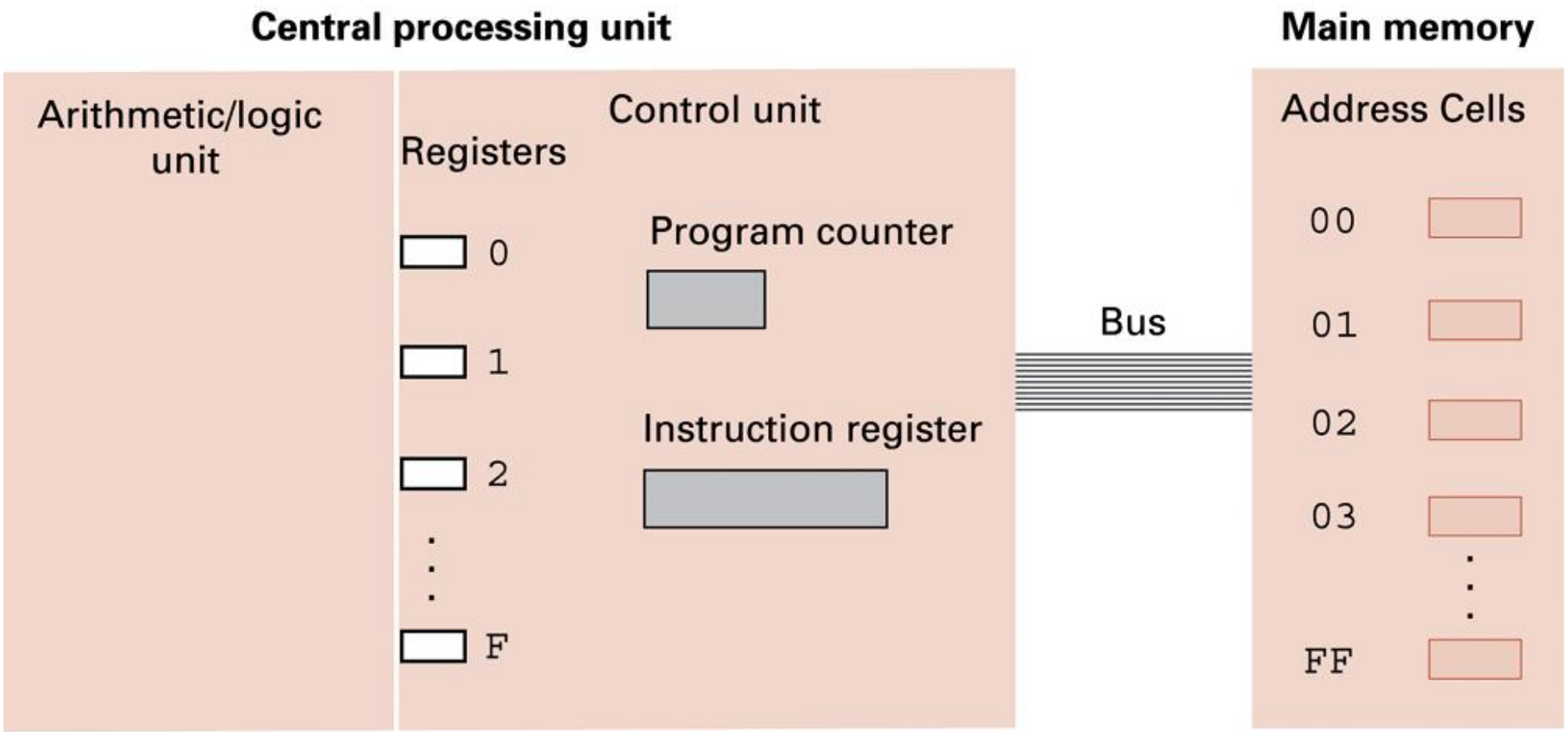
Step 2. LOAD another register with another value from memory.

Step 3. If this second value is zero, JUMP to Step 6.

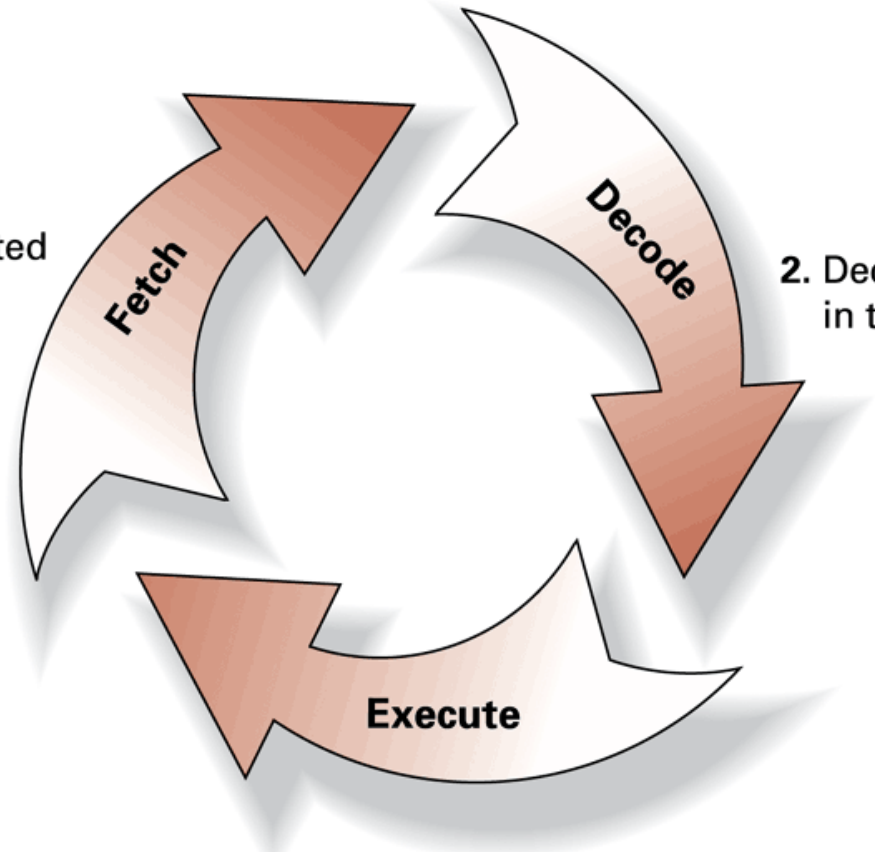
Step 4. Divide the contents of the first register by the second register and leave the result in a third register.

Step 5. STORE the contents of the third register in memory.

Step 6. STOP.


Central processing unit
Main memory
Arithmetic/logic unit
Control unit
Registers
0
1
2
F
Program counter
Instruction register
Bus
Address Cells
00
01
02
03
FF

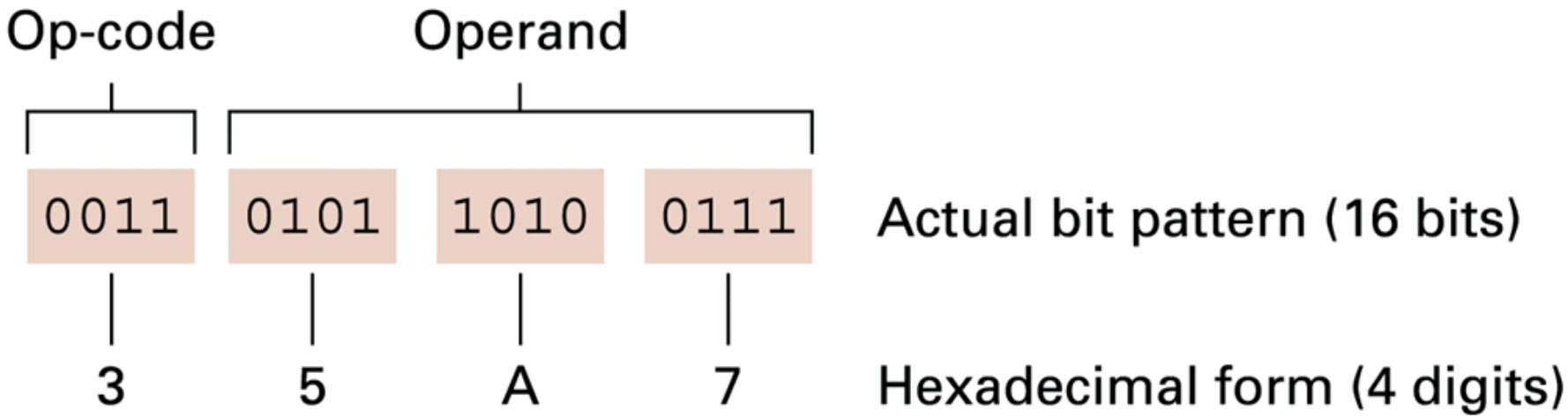
The machine cycle



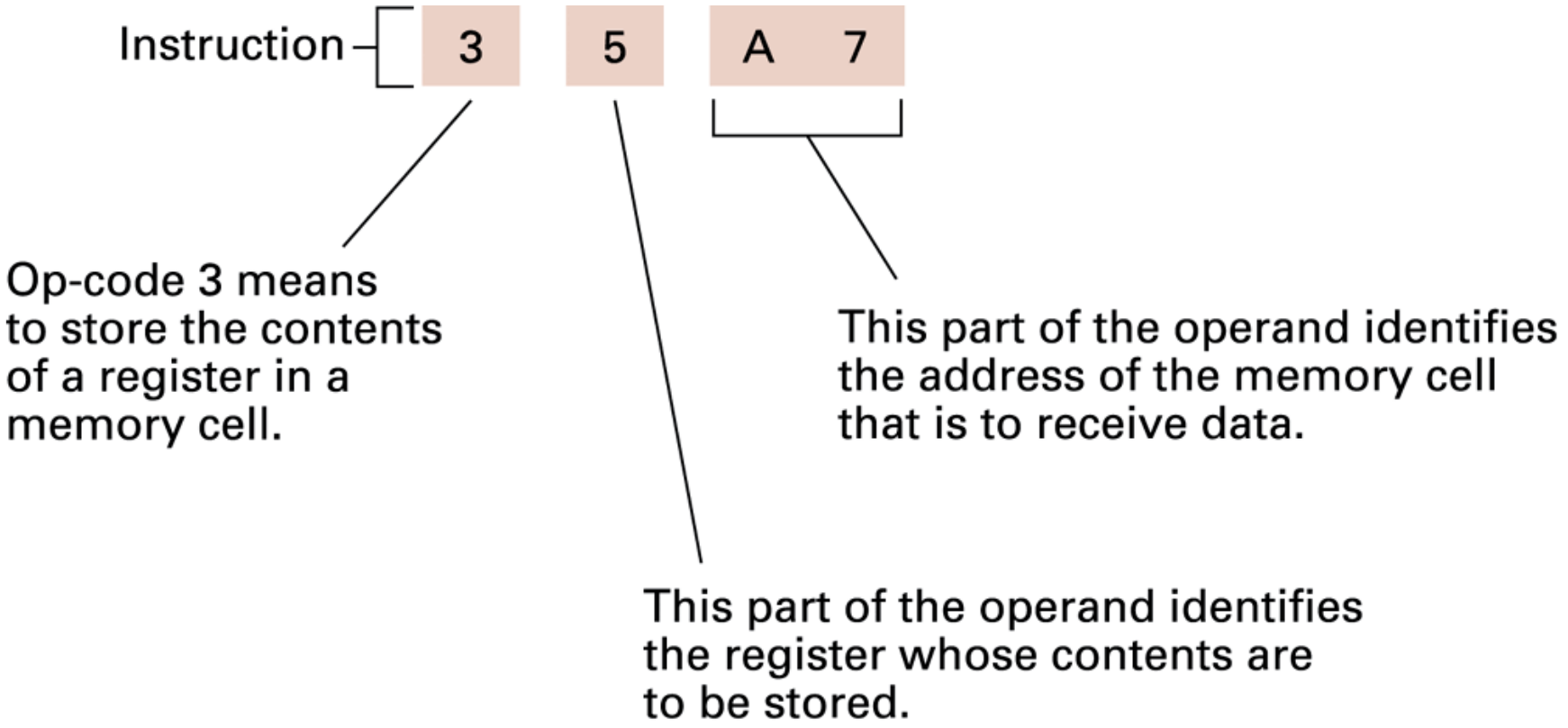
Program execution

- Controlled by two special-purpose registers
  - Program counter: address of next instruction
  - Instruction register: current instruction
- Steps performed by control unit
  - Fetch
  - Decode
  - Execute

- Parts of a machine instruction
  - Op-code: specifies which machine operation to execute
  - Operand: more detailed information about this operation



› Decoding the instruction 35A7



The composition of an instruction for the machine in Appendix C

Op-code	Operand	Description
1	RXY	LOAD the register R with the bit pattern found in the memory cell whose address is XY. *Example:* 14A3 would cause the contents of the memory cell located at address A3 to be placed in register 4.
2	RXY	LOAD the register R with the bit pattern XY. *Example:* 20A3 would cause the value A3 to be placed in register 0.
3	RXY	STORE the bit pattern found in register R in the memory cell whose address is XY. *Example:* 35B1 would cause the contents of register 5 to be placed in the memory cell whose address is B1.

4	0RS	MOVE the bit pattern found in register R to register S. *Example:* 40A4 would cause the contents of register A to be copied into register 4.
5	RST	ADD the bit patterns in registers S and T as though they were two's complement representations and leave the result in register R. *Example:* 5726 would cause the binary values in registers 2 and 6 to be added and the sum placed in register 7.
6	RST	ADD the bit patterns in registers S and T as though they represented values in floating-point notation and leave the floating-point result in register R. *Example:* 634E would cause the values in registers 4 and E to be added as floating-point values and the result to be placed in register 3.

7	RST	OR the bit patterns in registers S and T and place the result in register R. *Example:* 7CB4 would cause the result of ORing the contents of registers B and 4 to be placed in register C.
8	RST	AND the bit patterns in registers S and T and place the result in register R. *Example:* 8045 would cause the result of ANDing the contents of registers 4 and 5 to be placed in register 0.
9	RST	EXCLUSIVE OR the bit patterns in registers S and T and place the result in register R. *Example:* 95F3 would cause the result of EXCLUSIVE ORing the contents of registers F and 3 to be placed in register 5.
A	R0X	ROTATE the bit pattern in register R one bit to the right X times. Each time place the bit that started at the low-order end at the high-order end. *Example:* A403 would cause the contents of register 4 to be rotated 3 bits to the right in a circular fashion.

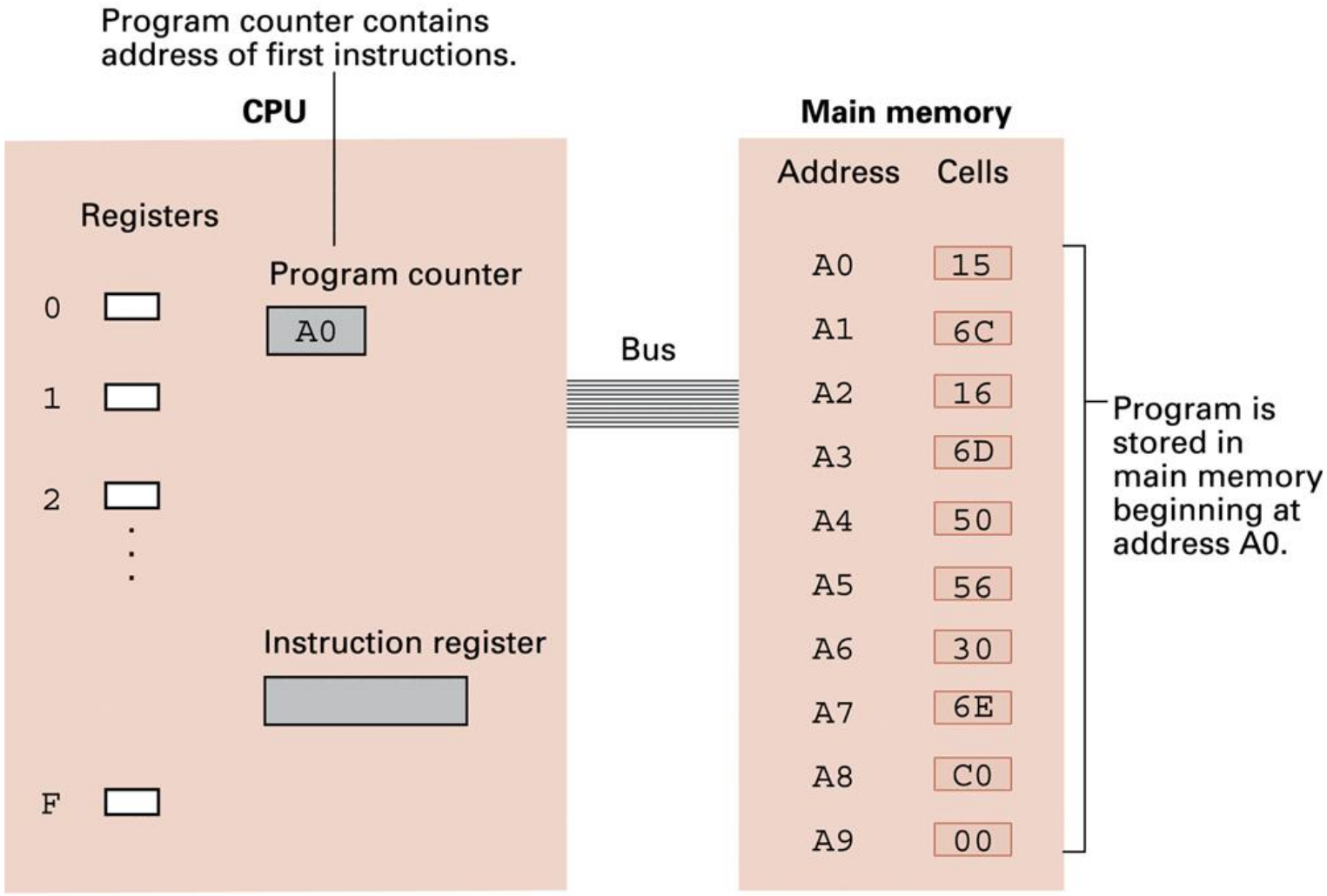
B	RXY	JUMP to the instruction located in the memory cell at address XY if the bit pattern in register R is equal to the bit pattern in register number 0. Otherwise, continue with the normal sequence of execution. (The jump is implemented by copying XY into the program counter during the execute phase.) *Example:* B43C would first compare the contents of register 4 with the contents of register 0. If the two were equal, the pattern 3C would be placed in the program counter so that the next instruction executed would be the one located at that memory address. Otherwise, nothing would be done and program execution would continue in its normal sequence.
C	000	HALT execution. *Example:* C000 would cause program execution to stop.

Arithmetic/Logic operations

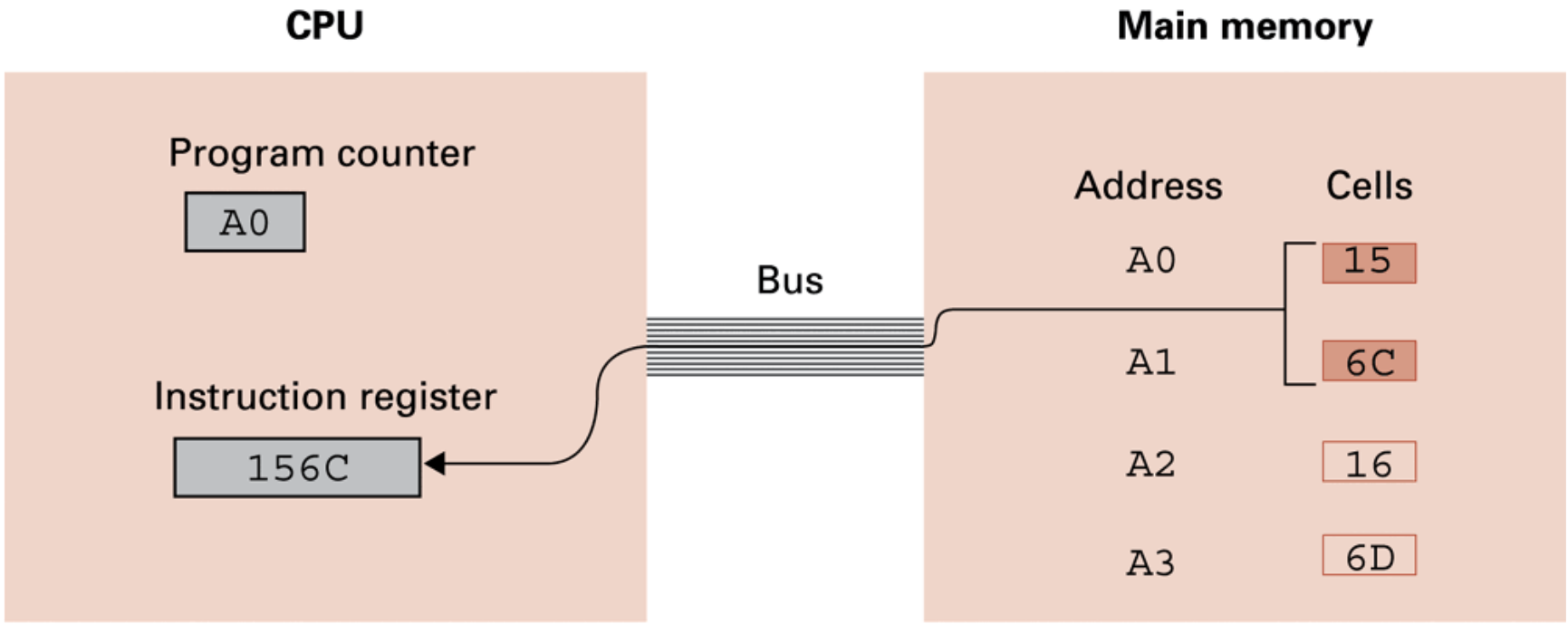
- Logic: AND(8), OR(7), XOR(9)
- Rotate and Shift: circular shift, logical shift, arithmetic shift
- Arithmetic: add, subtract, multiply, divide
  - Often separate instructions for different types of data

Encoded instructions	Translation
156C	Load register 5 with the bit pattern found in the memory cell at address 6C.
166D	Load register 6 with the bit pattern found in the memory cell at address 6D.
5056	Add the contents of register 5 and 6 as though they were two's complement representation and leave the result in register 0.
306E	Store the contents of register 0 in the memory cell at address 6E.
C000	Halt.

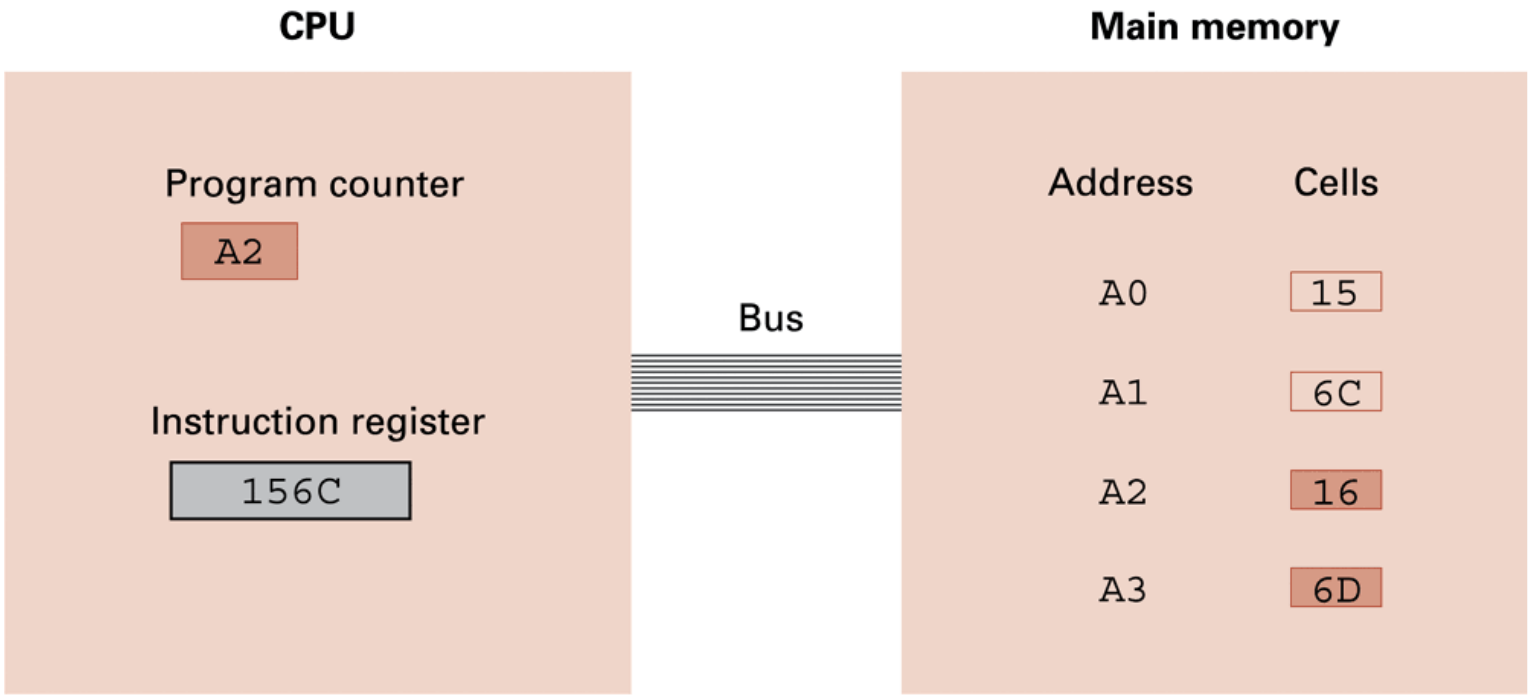
The program stored in main memory ready for execution



› Performing the fetch step of the machine cycle



a. At the beginning of the fetch step the instruction starting at address A0 is retrieved from memory and placed in the instruction register.



b. Then the program counter is incremented so that it points to the next instruction.

An example

- The following table shows a portion of a machine's memory containing a program written in the language described in the language description table. Answer the questions below assuming that the machine is started with its program counter containing 00.

address	content
00	21
01	0B
02	14
03	04
04	C0
05	00

  - What bit pattern will be in register 4 when the machine halts?
  - What bit pattern will be in register 1 when the machine halts?

address	content
00	20
01	05
02	21
03	01
04	22
05	04
06	51
07	12
08	B1
09	0C
0A	23
0B	12
0C	C0
0D	00

Answer the questions below assuming that the machine is started with its program counter containing 00.

1. What bit pattern will be in register 0 when the machine halts?
2. What bit pattern will be in register 1 when the machine halts?
3. What bit pattern will be in register 2 when the machine halts?
4. What bit pattern will be in register 3 when the machine halts?
5. What bit pattern will be in Program Counter when the machine halts?

› Q&A…